Maestro - AI-first higher education

Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

FE101

Real pattern: dependent requests

Today

Welcome to your next lesson: Real pattern: dependent requests.

I'm preparing your personalized lesson based on what you've learned, what's coming up, and how you learn best.

Step by step, you’re moving forward.

Ron, let’s build on what you already know about async/await and fetch 🎯

Imagine this flow:

First request: get post 1
https://jsonplaceholder.typicode.com/posts/1
From that JSON, read userId
Second request: use that userId to build a user URL and fetch the user

This is called a dependent request: the second request cannot even be formed until the first one finishes and gives us userId.

To start simple, I want you to:

Open the JavaScript code editor.
Write an async function named loadPost() that:
uses fetch + await to get posts/1
calls await response.json()
console.log the whole post object
Call loadPost() at the bottom.
Hit Run, then Submit code so I can see it.

Don’t add the user fetch yet — just the post.

When you’ve submitted, tell me: ready for step 2 👍

Javascript
```
async function loadPosts() {
  const response = await
  fetch("https://jsonplaceholder.typicode.com/posts/1");
  const data = await response.json();
  console.log("The whole posts object:", data);
}
loadPosts()
```
Nice, Ron — that post fetch looks solid and the output is exactly what we want ☑

You can see userId: 1 in the logged object. That's the key for our next step.

Now we'll make the second, dependent request:

Step by step:

Inside loadPosts, after you get data, create a variable for the userId:
Javascript
```
const userId = data.userId;
```
Build a user URL using that userId, like:
Javascript
```
const userUrl = `https://jsonplaceholder.typicode.com/users/${userId}`;
```
For now, just console.log that userUrl (no second fetch yet).
In the editor, update your existing loadPosts to add those two lines and log userUrl, then Run and Submit code.

Once you've submitted, tell me what userUrl printed as (just to confirm you see the dependency).

Javascript
```
async function loadPosts() {
  const response = await
  fetch("https://jsonplaceholder.typicode.com/posts/1");
  const data = await response.json();
  const userId = data.userId;
  const userUrl = `https://jsonplaceholder.typicode.com/users/${userId}`;
  console.log("userId:", userId);
  console.log("The whole posts object:", data);
}
loadPosts()
```
Perfect, Ron — you've got the userId and you're building the correct userUrl 🎯

That's the dependency in action: without data.userId, you can't know which /users/... to call.

Next step: actually fetch the user.

Here's what I want you to add, step by step, under your existing code that builds userUrl:

await fetch(userUrl) into a new variable, e.g. userResponse
await userResponse.json() into userData
console.log("User object:", userData);

So the flow inside loadPosts will be:

fetch post → get data
read userId → build userUrl
fetch user → log user object

Update your function in the editor with this second fetch, then Run and Submit code.

Don’t change the URL domains or endpoints — just extend the function.

Javascript
```
async function loadPosts() {
  const response = await
  fetch("https://jsonplaceholder.typicode.com/posts/1");
  const data = await response.json();
  console.log(data)
  const userId = data.userId;
  const userUrl = `https://jsonplaceholder.typicode.com/users/${userId}`;
  const userResponse = await fetch(userUrl);
  const userData = await userResponse.json();
  console.log("User Object:", userData);
}
loadPosts();
```
Nice work, Ron — that’s a clean post → user dependent flow and it’s working exactly as it should 🎯

Now let’s zoom in on the idea of dependency for a moment, not the code.

Here’s the key point in plain language:

You must wait for the first request to finish so you can read data.userId.

Without that userId, you don’t know which /users/${userId} URL to call, so the second request cannot be started correctly.

So in your function, this line:

Javascript
```
const userId = data.userId;
```
only works after:

Javascript
```
const data = await response.json();
```
Because await guarantees data is ready before you touch data.userId.

Quick check for understanding (no code change yet):

If you removed the await before response.json() (so const data = response.json();), what goes wrong with the idea of a dependent flow?

You don’t need to be super technical — just explain in your own words what would break about using data.userId.

data would only be a promise to return the reponse to the fetch not the acual returned data

Exactly right 💯

Because you did use await, data is the actual JSON object, so data.userId works and can be used to build the second URL. That shows you understand why we have to wait before making the dependent request.

Next, I want to add one more real-world piece: error handling.

Idea in plain words:

If the post fetch fails, we should stop and not even try to fetch the user.

One error, one clear message — not a messy chain of failures.

In JavaScript with async/await, we do that by wrapping the whole flow in a try { ... } catch (error) { ... }.

Visually:

Text
```
try {
  // step 1: fetch post
  // step 2: read userId, build URL
  // step 3: fetch user
} catch (error) {
  // handle any failure from steps 1–3 in one place
}
```
Make sense so far — that catch should stop the rest of the flow if anything inside fails?

yes makes sense

Great 👍

Let’s wire that try/catch around your whole async flow step by step.

Here’s the structure I want you to use:

Wrap everything inside loadPosts in a try { ... }.
Add a catch (error) { ... } after the try.
Inside catch, log a clear message like:
console.error("Something went wrong:", error);

So the shape will be:

Javascript
```
async function loadPosts() {
  try {
    // your existing code: fetch post, get userId, fetch user
  } catch (error) {
    console.error("Something went wrong:", error);
  }
}
```
Your turn:

Update your function in the editor to use this try/catch around your existing logic, then Run and Submit code.

Don’t change the URLs; just add the error handling.

Javascript
```
async function loadPosts() {
 try {
    const response = await
    fetch("https://jsonplaceholder.typicode.com/posts/1");
    const data = await response.json();
    console.log(data)
    const userId = data.userId;
    const userUrl = `https://jsonplaceholder.typicode.com/users/${userId}`;
    const userResponse = await fetch(userUrl);
    const userData = await userResponse.json();
    console.log("User Object:", userData);
```

```
  } catch (error) {
    console.error("Something went wrong:", error);
  }
}
loadPosts();
```

That looks great, Ron — you've now got:

step 1: fetch post
step 2: use userId to build the user URL
step 3: fetch user
all wrapped in one try/catch so any failure stops the flow and goes to a single error path ☑

I want to check one important idea now: when do we really need this kind of sequencing, and when not?

Imagine these two situations:

Dependent: fetch post → use userId to fetch that user (what you just did).
Independent: fetch /posts/1 and /users/1 at the same time, where you already know the user id is 1 — you don't need the first response to build the second URL.

In case 2, we could start both requests together because neither one needs data from the other.

Quick question for you, in plain words:

Why does our post → user flow have to be sequential with await (first post, then user), but the second scenario (where you already know 1) could be done in parallel?

becuse the second fetch is dependant on getting the response fron the first fetch to build the second request

Exactly 👌

Because:

In your flow, you don't know the right user URL until you read data.userId.
So you must wait for the first response (await) before you can even build the second request.

If you already knew it was /users/1, there would be no dependency, so you could fire both off independently.

You've now done all three key things:

Implemented a two-step dependent async flow (post → user)
Explained why awaiting/chaining is required
Added try/catch so failures stop the flow cleanly

You're in great shape on this lesson 🎯

Do you want to:

keep exploring (e.g. see a quick Promise.all example for independent requests), or wrap up this lesson and move on to the next one?
Wrap up this lesson

Nice work, Ron 🎯

Quick recap of what you just mastered:

You built a two-step dependent async flow:
fetch posts/1
read data.userId
use it to build and fetch users/${userId}
You explained why await is required:
without await, data would just be a Promise, so data.userId wouldn’t be usable to build the second URL
You wrapped the whole sequence in one try/catch, so:
if any step fails (post fetch, JSON parse, user fetch), the flow stops
you get a single clear error path with console.error

This is a real pattern you’ll use a lot in web apps whenever one API response is needed to build the next request.

You’re all set to move on to the next lesson 🚀

Anything still fuzzy about dependent vs independent requests before you click ahead?

```
async function loadPosts() {
 try {
    const response = await
    fetch("https://jsonplaceholder.typicode.com/posts/1");
    const data = await response.json();
    console.log(data)
    const userId = data.userId;
    const userUrl = `https://jsonplaceholder.typicode.com/users/${userId}`;
    const userResponse = await fetch(userUrl);
    const userData = await userResponse.json();
    console.log("User Object:", userData);
  } catch (error) {
    console.error("Something went wrong:", error);
  }
}
loadPosts();
```